Elastic Net Factor Regression

Coordinate descent, hyperparameter and factor selection, validation and the block bootstrap: theory, proofs and the `nabia` implementation

nabia documentation

October 2026

Abstract

This note documents the regression module of `nabia`: the elastic net solver (`ElasticNet`) and the `RegressionEngine`. The engine regresses a target series on a set of candidate factors. It chooses how many factors to use and which ones, the penalty λ and the mix α, and measures the uncertainty of the result.

We first derive the elastic net from its optimality conditions:

- the coordinate descent update and soft thresholding;
- the smallest penalty that zeroes every coefficient, $\lambda_{\max}$;
- the role of the intercept and of standardization;
- the grouping effect that makes the elastic net stable with correlated factors;
- the effective degrees of freedom.

We then describe the engine:

- the chronological train/test split;
- time-series cross-validation (blocked K-fold and walk-forward, with a gap);
- the one-standard-error rule;
- the in-sample criteria (BIC, AIC) used when there is no test set;
- exhaustive and forward subset search;
- the moving block bootstrap.

The last sections map each step to the code, show how to use it from C++ and from Python, and list the tests and the limitations.

Contents

1 Problem and notation **2**

2 The elastic net **2**

2.1 Objective 2
2.2 Intercept and standardization 3
2.3 Optimality conditions 3
2.4 The coordinate update 3
2.5 The smallest penalty that zeroes everything 4
2.6 Lambda grid and warm starts 4
2.7 Grouping effect: why the elastic net for correlated factors 5
2.8 Effective degrees of freedom 5

3 The regression engine **6**
3.1 Overview . . . 6
3.2 Chronological split . . . 6
3.3 Choosing λ and α: time-series cross-validation . . . 6
3.4 Scores . . . 7
3.5 Subset search . . . 7
3.6 Refit on the full sample . . . 8
3.7 Moving block bootstrap . . . 8
3.8 Parallelism and determinism . . . 9
3.9 Complexity . . . 9

4 Implementation **9**
4.1 Code map . . . 9
4.2 Parameters . . . 10
4.3 Usage from C++ . . . 10
4.4 Usage from Python . . . 11
4.5 Worked example: recovering a known factor mix . . . 11

5 Verification **12**

6 Limitations **12**

1 Problem and notation

We observe a target y_t (for example a fund's daily return) and N candidate factors $x_{t1}, \dots, x_{tN}$ (market, rate, commodity returns) for $t = 1, \dots, T$, in time order. We look for a sparse linear model

$$y_t = b_0 + \sum_{j \in S} b_j x_{tj} + \varepsilon_t, \qquad S \subseteq \{1, \dots, N\}, \quad k_{\min} \le |S| \le k_{\max},$$

that *extrapolates*, that is, predicts y well on dates after those used to fit it. When only in-sample explanation is wanted, the same machinery chooses the model by an information criterion instead.

Throughout, $X \in \mathbb{R}^{n \times p}$ is the design of the rows and columns at hand (a training window, a fold, a subset), x_j its j-th column, $\|\cdot\|$ the Euclidean norm and $\mathcal{S}(z, t) = \operatorname{sign}(z) \max(|z| - t, 0)$ the soft-thresholding operator.

2 The elastic net

2.1 Objective

For $\lambda \ge 0$ and $\alpha \in [0, 1]$, the elastic net in the glmnet parametrization solves

$$\min_{b_0, b} \underbrace{\frac{1}{2n} \|y - b_0 \mathbf{1} - Xb\|^2}_{\text{fit}} + \lambda \Big(\alpha \|b\|_1 + \frac{1 - \alpha}{2} \|b\|_2^2 \Big). \tag{1}$$

- $\alpha = 1$ is the *lasso*: the ℓ_1 penalty sets coefficients exactly to zero, so it selects factors.
- $\alpha = 0$ is *ridge* regression: the ℓ_2 penalty shrinks coefficients but never zeroes them.
- $\lambda = 0$ is ordinary least squares.

We write $\ell_1 = \lambda\alpha$ and $\ell_2 = \lambda(1 - \alpha)$ for the two penalty weights. The intercept is never penalized.

2.2 Intercept and standardization

Proposition 2.1 (Intercept)**.** *For any b, the optimal intercept in* (1) *is $b_0 = \bar{y} - \bar{x}^\top b$, where $\bar{y}$ and $\bar{x}$ are the means of y and of the columns of X. Substituting it leaves problem* (1) *without intercept on the centred data $\tilde{y} = y - \bar{y}\mathbf{1}$, $\tilde{X} = X - \mathbf{1}\bar{x}^\top$.*

Proof. The objective is a convex quadratic in b_0 with derivative $-\frac{1}{n}\mathbf{1}^\top(y - b_0\mathbf{1} - Xb) = -(\bar{y} - b_0 - \bar{x}^\top b)$, which vanishes at the stated value. Then $y - b_0\mathbf{1} - Xb = \tilde{y} - \tilde{X}b$. □

Standardization. Write $s_j^2 = \frac{1}{n}\|\tilde{x}_j\|^2$ (the population variance of factor j). With `standardize`, the solver works on $z_j = \tilde{x}_j/s_j$ and finds coefficients β_j for those columns. The coefficients in original units are

$$b_j = \beta_j/s_j, \qquad b_0 = \bar{y} - \bar{x}^\top b.$$

Equivalently, the penalty in original units is $\lambda(\alpha\sum_j s_j|b_j| + \frac{1-\alpha}{2}\sum_j s_j^2 b_j^2)$: each factor is charged in proportion to its volatility. A factor's units, or its leverage (a 2x version of an index), therefore do not change whether it is selected. Without `standardize`, the penalty applies to b directly.

Constant factors. A column whose centred mean square is below $10^{-24}\max(1, \frac{1}{n}\|x_j\|^2)$ is constant up to rounding. It is set to zero, and its coefficient is fixed at 0. The relative threshold is needed because centring a constant column leaves rounding noise of the size of its magnitude.

In the working units, let $v_j = \frac{1}{n}\|z_j\|^2$. Then $v_j = 1$ with standardization, $v_j = s_j^2$ without it, and $v_j = 0$ for a constant column. The problem the solver works on is

$$\min_\beta \; F(\beta) = \frac{1}{2n}\|\tilde{y} - Z\beta\|^2 + \ell_1\|\beta\|_1 + \frac{\ell_2}{2}\|\beta\|_2^2. \tag{2}$$

2.3 Optimality conditions

F is convex, as the sum of a convex quadratic and two norms. With $\ell_2 > 0$ it is strictly convex, so the minimizer is unique. A point β minimizes F if and only if $0 \in \partial F(\beta)$, that is, with residual $r = \tilde{y} - Z\beta$:

$$\frac{1}{n}z_j^\top r - \ell_2\beta_j \begin{cases} = \ell_1\,\mathrm{sign}(\beta_j) & \beta_j \neq 0, \\ \in [-\ell_1, \ell_1] & \beta_j = 0. \end{cases} \tag{3}$$

An active factor has its correlation with the residual pinned at $\pm\ell_1$, after the ridge shrinkage. An inactive factor has a correlation with the residual below the threshold. The tests check (3) directly on solutions.

2.4 The coordinate update

Proposition 2.2 (Soft thresholding)**.** *Fix every coefficient but β_j, with $v_j > 0$, and let r be the current residual. The minimizer of F over β_j is*

$$\beta_j^{new} = \frac{\mathcal{S}(\rho_j, \ell_1)}{v_j + \ell_2}, \qquad \rho_j = \frac{1}{n}z_j^\top r + v_j\beta_j^{old}.$$

Proof. Let $r^{(j)} = r + z_j\beta_j^{\mathrm{old}}$ be the partial residual, the residual without factor j. As a function of β_j, up to constants,

$$F = \frac{1}{2n}\|r^{(j)} - z_j\beta_j\|^2 + \ell_1|\beta_j| + \frac{\ell_2}{2}\beta_j^2 = \frac{v_j + \ell_2}{2}\beta_j^2 - \rho_j\beta_j + \ell_1|\beta_j| + \text{const},$$

because $\frac{1}{n}z_j^\top r^{(j)} = \frac{1}{n}z_j^\top r + v_j\beta_j^{\mathrm{old}} = \rho_j$. This is a scalar problem $\min_u \frac{a}{2}u^2 - \rho u + \ell_1|u|$ with $a > 0$. Its subgradient condition is $au - \rho + \ell_1\partial|u| \ni 0$:

- if $u > 0$, then $u = (\rho - \ell_1)/a$, which needs $\rho > \ell_1$;
- if $u < 0$, then $u = (\rho + \ell_1)/a$, which needs $\rho < -\ell_1$;
- otherwise $u = 0$ is optimal, because $|\rho| \le \ell_1$ means $0 \in -\rho + [-\ell_1, \ell_1]$.

In every case $u = \mathcal{S}(\rho, \ell_1)/a$. □

After the update the residual is corrected in $O(n)$: $r \leftarrow r - z_j(\beta_j^{\text{new}} - \beta_j^{\text{old}})$. A sweep over all factors costs $O(np)$, and no matrix is ever formed or inverted.

Convergence. The non-smooth part of F is separable across coordinates, and the smooth part is a convex quadratic. Cyclic coordinate descent therefore converges to a minimizer (Tseng, 2001). Each update cannot increase F, because it minimizes F exactly along one coordinate.

Stopping rule. A change Δ_j of coordinate j moves the fitted values by $z_j\Delta_j$, whose mean square is $v_j\Delta_j^2$. The solver stops when, over a full sweep,

$$\max_j v_j\Delta_j^2 < \tau \cdot \tfrac{1}{n}\|\tilde{y}\|^2,$$

that is, when no coordinate moves the fit by more than a fraction τ (`tolerance`, default 10^{-7}) of the target's variance. The rule is the same in standardized and in raw units, and does not depend on the scale of y.

Active-set iterations. After a full sweep, the solver sweeps only the non-zero coefficients until they settle. Then it checks with a new full sweep. It stops when a *full* sweep meets the rule, so a factor that should enter is never missed. Inactive factors usually stay at zero, so most sweeps cost $O(n|A|)$ instead of $O(np)$.

2.5 The smallest penalty that zeroes everything

Proposition 2.3 ($\lambda_{\max}$)**.** *For $\alpha > 0$, $\beta = 0$ solves (2) if and only if*

$$\lambda \ge \lambda_{\max} = \frac{\max_j |z_j^\top \tilde{y}|/n}{\alpha}.$$

Proof. At $\beta = 0$, $r = \tilde{y}$ and every coefficient is inactive. By (3), 0 is optimal if and only if $|\frac{1}{n}z_j^\top \tilde{y}| \le \ell_1 = \lambda\alpha$ for every j. □

For ridge ($\alpha = 0$) no finite λ zeroes the coefficients. As in glmnet, α is floored at 10^{-3} in this formula, so ridge also gets a finite path that starts strongly shrunk.

The code computes $\lambda_{\max}$ with the same arithmetic as the first coordinate update and multiplies it by $1 + 10^{-12}$. Without that factor, rounding in the division can leave $\lambda\alpha$ a few ulps below $|\rho_j|$, which leaves a coefficient of order 10^{-17} instead of an exact zero.

2.6 Lambda grid and warm starts

The automatic grid has L (`n_lambdas`) values, log-spaced from $\lambda_{\max}$ down to $r\lambda_{\max}$ (`lambda_min_ratio`):

$$\lambda_i = \lambda_{\max}\, r^{\,i/(L-1)}, \qquad i = 0, \dots, L-1.$$

The path is solved in this decreasing order, and each solution starts from the previous one (a *warm start*). The first solution is $\beta = 0$ exactly. Each later solution is close to the previous one, so it takes a few sweeps. A user-supplied grid is sorted in decreasing order for the same reason.

2.7 Grouping effect: why the elastic net for correlated factors

The lasso picks one factor out of a group of highly correlated ones, somewhat arbitrarily, and may switch between them from sample to sample. The ridge term stops this.

Proposition 2.4 (Grouping)**.** *Let the factors be standardized ($\frac{1}{n}\|z_j\|^2 = 1$), let $\ell_2 > 0$, and let factors i, j have correlation ρ_{ij} and coefficients of the same sign. Then*

$$|\beta_i - \beta_j| \le \frac{\|\tilde{y}\|}{\sqrt{n}\,\ell_2}\sqrt{2(1-\rho_{ij})}.$$

Proof. Both coefficients satisfy (3) with the same sign s: $\frac{1}{n}z_i^\top r - \ell_2\beta_i = \ell_1 s = \frac{1}{n}z_j^\top r - \ell_2\beta_j$. Hence $\ell_2(\beta_i - \beta_j) = \frac{1}{n}(z_i - z_j)^\top r$, and by Cauchy–Schwarz $|\beta_i - \beta_j| \le \frac{1}{n\ell_2}\|z_i - z_j\|\|r\|$. Here $\|z_i - z_j\|^2 = 2n(1-\rho_{ij})$. Also $F(\hat{\beta}) \le F(0)$ gives $\|r\| \le \|\tilde{y}\|$. □

As $\rho_{ij} \to 1$, the two coefficients are forced together, and they share the loading instead of one taking it all. In the factor replication example (S&P 500 against Nasdaq 100, correlation 0.93), $\alpha < 1$ keeps the split stable across bootstrap samples.

2.8 Effective degrees of freedom

The information criteria of §3.4 need the number of parameters an elastic net fit really uses. Penalized coefficients count for less than one.

Proposition 2.5 (Degrees of freedom)**.** *Let A be the active set of the solution and Z_A its columns. For a y at which the active set and the signs are locally constant,*

$$\mathrm{df} = \mathbb{1}[\textit{intercept}] + \operatorname{tr}\bigl(Z_A(Z_A^\top Z_A + n\ell_2 I)^{-1}Z_A^\top\bigr) = \mathbb{1}[\textit{intercept}] + \sum_i \frac{d_i}{d_i + n\ell_2},$$

where the d_i are the eigenvalues of $Z_A^\top Z_A$.

Proof. For a fit $\hat{y} = g(y)$, Stein's lemma gives the unbiased degrees of freedom $\mathrm{df} = \sum_t \partial\hat{y}_t/\partial y_t = \operatorname{tr}(\partial g/\partial y)$ (Efron, 2004). On the active set, (3) with fixed signs s_A reads $\frac{1}{n}Z_A^\top(\tilde{y} - Z_A\beta_A) - \ell_2\beta_A = \ell_1 s_A$, so

$$\beta_A = (Z_A^\top Z_A + n\ell_2 I)^{-1}(Z_A^\top\tilde{y} - n\ell_1 s_A).$$

The second term does not depend on y locally. Hence $\partial(Z_A\beta_A)/\partial\tilde{y} = H = Z_A(Z_A^\top Z_A + n\ell_2 I)^{-1}Z_A^\top$. With an intercept, $\hat{y} = \bar{y}\mathbf{1} + Z_A\beta_A$ and $\tilde{y} = (I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top)y$. The centred columns are orthogonal to $\mathbf{1}$, so the trace is $1 + \operatorname{tr} H$. Diagonalizing $Z_A^\top Z_A = UDU^\top$ gives $\operatorname{tr} H = \operatorname{tr}\bigl(D(D + n\ell_2 I)^{-1}\bigr)$. □

Special cases:

- the lasso ($\ell_2 = 0$) with Z_A of full rank gives $\mathrm{df} = |A|$ (+1), the result of Zou, Hastie and Tibshirani (2007);
- ridge gives the familiar $\sum d_i/(d_i + n\ell_2)$;
- OLS gives the number of columns.

Eigenvalues below $10^{-10}\max_i d_i$ are treated as zero, which handles a rank-deficient active set.

3 The regression engine

3.1 Overview

Given $y \in \mathbb{R}^T$, $X \in \mathbb{R}^{T \times N}$ and the parameters, the engine runs these steps:

1. **Split** the rows in time order into training and test sets (§3.2).
2. For every candidate subset S (§3.5):
   (a) choose (α, λ) by time-series cross-validation on the training rows (§3.3);
   (b) fit the elastic net on all training rows at that (α, λ);
   (c) score it by the test MSE, or by an in-sample criterion when there is no test set (§3.4).
3. **Keep** the subset with the lowest score. Optionally refit it on every row (§3.6) and bootstrap it (§3.7).

3.2 Chronological split

The training set is the first

$$n = \min\left(T, \lfloor fT + 10^{-9} \rfloor\right)$$

rows, with $f =$ `train_fraction`. The test set is the remaining $T - n$ rows. The 10^{-9} makes 0.7×10 count as 7 despite rounding. Rows are never shuffled.

- The test set lies after every training row, so its error measures *extrapolation*: the model faces a period it has never seen, with its own regimes and correlations.
- With $f = 1$ there is no test set, and the engine works *in sample*: subsets are compared by `in_sample_criterion`.

3.3 Choosing λ and α: time-series cross-validation

Grid. For each α_a of `alphas`, the engine builds a λ grid $\lambda_{a,0} > \cdots > \lambda_{a,L-1}$. It is the automatic grid from $\lambda_{\max}$ of the subset's training rows, or the user's `lambdas`. The grid differs by α, because $\lambda_{\max} \propto 1/\alpha$, so cells are compared by position. If the grid has a single point, there is nothing to choose, and cross-validation is skipped.

Folds. The n training rows are cut into contiguous blocks with boundaries $\lfloor kn/B \rfloor$. With gap g (`validation_gap`):

Blocked K-fold ($B = K$ blocks). Fold k validates on block k and trains on every row before the block minus g, and every row after it plus g.

Walk-forward ($B = K + 1$ blocks). Fold $k = 1, \ldots, K$ validates on block k and trains on the rows before it minus g. Training always precedes validation, as in live use. The cost is smaller training sets in the first folds.

The gap removes the rows next to the validation block from training. Returns are serially dependent, through overlapping horizons or volatility clustering, so a training row adjacent to a validation row leaks information about it. A fold with no validation rows, or with fewer than 2 training rows, is an error.

Estimate and standard error. For every fold k and grid cell (a, i), the path for α_a is fitted on the fold's training rows. Its validation MSE is $m_k(a, i)$. With K folds:

$$\mathrm{CV}(a,i) = \frac{1}{K}\sum_k m_k(a,i), \qquad \mathrm{SE}(a,i) = \sqrt{\frac{1}{K}\cdot\frac{1}{K-1}\sum_k \big(m_k(a,i) - \mathrm{CV}(a,i)\big)^2}.$$

These are the mean of the fold errors and its standard error, treating the folds as independent replicates. The chosen cell is $(a^*, i^*) = \arg\min \mathrm{CV}$, with ties going to the first cell, so the result is deterministic.

One-standard-error rule. With `one_standard_error_rule`, the engine stays on row a^* and takes the *largest* λ whose error is within one standard error of the minimum:

$$i^{**} = \min\{\, i \le i^* :\ \mathrm{CV}(a^*, i) \le \mathrm{CV}(a^*, i^*) + \mathrm{SE}(a^*, i^*)\,\}.$$

The CV minimum is a noisy estimate. A model that is statistically as good but more strongly penalized (sparser, more stable) is preferred (Hastie, Tibshirani and Friedman, 2009, §7.10).

Final fit of the subset. The subset is refitted on all n training rows along the grid of α_{a^*}, from $\lambda_{a^*,0}$ down to the chosen value. This warm-starts it exactly as during validation.

3.4 Scores

Out of sample $(T > n)$. The score is the test MSE, $\frac{1}{T-n}\sum_{t>n}(y_t - \hat{y}_t)^2$. The engine also reports the test R^2:

$$R^2_{\text{test}} = 1 - \frac{\sum_{t>n}(y_t - \hat{y}_t)^2}{\sum_{t>n}(y_t - \bar{y}_{\text{train}})^2}.$$

It is measured against the *training* mean, the forecast available without the model (Campbell and Thompson, 2008). It is negative when the model does worse than that mean.

In sample $(T = n)$. The score is an information criterion on the training rows. Under Gaussian errors, the log-likelihood maximized over the noise variance is $-2\log\hat{L} = n\log(2\pi\,\mathrm{MSE}) + n$. Dropping the constants, which are the same for every model:

$$\mathrm{BIC} = n\log\mathrm{MSE} + \mathrm{df}\log n, \qquad \mathrm{AIC} = n\log\mathrm{MSE} + 2\,\mathrm{df},$$

with the effective df of Proposition 2.5.

- BIC is consistent: with a true sparse model it selects it with probability $\to 1$.
- AIC targets prediction error and admits more factors.
- `MSE` alone never pays for a factor. Among nested least-squares fits it always prefers the largest subset. It is there for comparison.

3.5 Subset search

With $k_{\min} =$ `min_factors` and $k_{\max} =$ `max_factors` (0 meaning N):

Exhaustive evaluates every subset of each size, in lexicographic order, which is $\sum_{k=k_{\min}}^{k_{\max}}\binom{N}{k}$ subsets. For each size it keeps the subset with the lowest score. For example, $N = 14$ and sizes 1–4 give $14 + 91 + 364 + 1001 = 1470$ subsets.

Forward is greedy. Starting from the empty set, step $k = 1, \ldots, k_{\max}$ adds the remaining factor whose addition gives the lowest score. That costs $\sum_{k=1}^{k_{\max}} (N - k + 1)$ evaluations, which is linear in N for a fixed $k_{\max}$. The best subset of size k is recorded for $k \geq k_{\min}$. Forward search can miss the best subset when factors only help together, but it reaches it in the common case of nested good models.

Auto uses Exhaustive if the count above is at most `max_exhaustive_subsets` (default 10 000), and Forward otherwise. The count is computed in floating point and saturates, so it never overflows.

The selected model is the best of the per-size winners (`best_by_size`), by score. Every evaluation is independent, so the subsets of one size (exhaustive) or one step (forward) are evaluated in parallel (§3.8).

Remark 3.1 (Selection on the test set). Choosing among many subsets by their test error makes the winner's test error an *optimistic* estimate of its error on new data, because the minimum of many noisy errors is biased down. A factor can win by noise. The bootstrap of §3.7 helps detect this: a spurious factor's interval contains zero. For a clean final number, keep a later period out of the engine entirely.

3.6 Refit on the full sample

With `refit_on_full_sample` and a test set, the chosen factors are refitted on all T rows at the chosen (α, λ). The reported coefficients and intercept are then the refit, which uses the most recent data. The predictions, residuals and scores remain those of the training-rows model, so the test rows stay out of sample.

3.7 Moving block bootstrap

With $B =$ `bootstrap_samples` > 0, the engine estimates the sampling distribution of the selected model's coefficients. Each sample $b = 1, \ldots, B$:

1. draws block starts s uniformly in $\{0, \ldots, n - \ell\}$, with $\ell =$ `bootstrap_block_size` $\leq n$, and concatenates the blocks of rows $s, \ldots, s + \ell - 1$ until there are n rows (the last block is truncated);
2. refits the elastic net on those rows, for the selected factors, at the selected (α, λ);
3. records the coefficients, the intercept and the MSE of that model on the test rows.

With $\ell = 1$ this is the ordinary i.i.d. bootstrap. Daily returns are not independent: their volatility clusters. Blocks keep the dependence within ℓ consecutive rows, and the moving block bootstrap is consistent for weakly dependent series when $\ell \to \infty$ and $\ell / n \to 0$ (Künsch, 1989). A block of about a month (20 rows) is a reasonable choice for daily data.

Summaries. For each selected factor, with draws $c_1, \ldots, c_B$:

- the mean, and the standard error $\sqrt{\frac{1}{B-1} \sum_b (c_b - \bar{c})^2}$;
- the percentile interval at level γ (`confidence_level`): the empirical quantiles at $q = (1 - \gamma)/2$ and $1 - q$. With the sorted draws $c_{(0)} \leq \cdots \leq c_{(B-1)}$, the quantile at q interpolates linearly at position $q(B - 1)$;
- the *selection frequency*, the fraction of samples with $c_b \neq 0$. It is meaningful for $\alpha > 0$; for ridge every coefficient is non-zero, so it is always 1;
- the intercept's mean and standard error, and the mean and percentile interval of the test MSE.

What the bootstrap does not cover. The resamples refit the *selected* subset at the *selected* (α, λ). The intervals are conditional on that choice: they measure estimation noise, not the uncertainty of selecting the factors. Redoing the whole search on every resample would cost B times the search.

Reproducibility. Sample b uses its own Mersenne Twister, seeded with `std::seed_seq` from the 64-bit `bootstrap_seed` and b. The draws do not depend on which thread runs which sample.

3.8 Parallelism and determinism

Subset evaluations and bootstrap samples run on `nabia::detail::parallel_for` with `num_threads` threads (0 = all cores). Every task writes only its own slot. Selections use a sequential argmin with first-index tie-breaking, and random draws are seeded per sample. Results are therefore bit-identical for every thread count, and a test checks this.

3.9 Complexity

One elastic net fit along an L-point path costs about $O(\text{sweeps} \cdot n|S|)$. One subset evaluation costs $|\text{alphas}| \times (K + 1)$ path fits. The search multiplies that by the number of subsets, and the bootstrap adds B single fits. In the factor replication example (14 factors, sizes 1–4 exhaustive, 3 alphas × 30 lambdas, 5 folds, about 2 600 training rows, 500 bootstrap samples), the whole example script, with its other variants, runs in about 4 s on a laptop.

4 Implementation

4.1 Code map

Paths are relative to the C++ project, `cpp/`.

Concept	Code
Parameters, fit, class	`include/nabia/regression/ElasticNet.h`: `ElasticNetParams`, `ElasticNetFit`, `ElasticNet`
Centring, scaling (§2.2)	`src/regression/ElasticNet.cpp`: `standardize_data`
Coordinate descent (Prop. 2.2)	`CoordinateDescent::solve`, `CoordinateDescent::sweep`, `soft_threshold`
$\lambda_{\max}$, grid (Prop. 2.3)	`nabia::regression::lambda_max`, `lambda_grid`
Path and warm starts	`nabia::regression::elastic_net_path`
Degrees of freedom (Prop. 2.5)	`degrees_of_freedom`
Back to original units	`make_fit`
Engine parameters and results	`include/nabia/regression/RegressionEngine.h`
Split, search, final model	`src/regression/RegressionEngine.cpp`: `run_regression_engine`
Folds (§3.3)	`make_folds`
CV, 1-SE rule, scores	`evaluate`, `in_sample_criterion`, `r_squared`
Subset search (§3.5)	`exhaustive_search`, `forward_search`, `count_subsets`
Bootstrap (§3.7)	`bootstrap`, `percentile`
Python bindings	`bindings/src/bind_regression.cpp`
pandas interface	`bindings/nabia_python_bridge/frames.py`: `regress`
Tests	`tests/test_regression.cpp`, `../python/tests/bridge/test_regression.py`
Worked example	`../python/examples/01_factor_replication.py`

4.2 Parameters

Field (`RegressionEngineParams`)	Default	Meaning
`min_factors`, `max_factors`	1, 0	subset sizes $k_{\min}..k_{\max}$; $k_{\max} = 0$ means all
`search`	`Auto`	`Exhaustive`, `Forward` or `Auto`
`max_exhaustive_subsets`	10 000	Auto: largest exhaustive search
`train_fraction`	0.7	f; 1 = in sample
`in_sample_criterion`	`BIC`	`BIC`, `AIC` or `MSE` (when $f = 1$)
`refit_on_full_sample`	false	report coefficients refitted on all rows
`alphas`	0, .25, .5, .75, 1	mixes α to try
`lambdas`	empty	fixed grid; empty = automatic
`n_lambdas`, `lambda_min_ratio`	50, 10^{-3}	automatic grid
`validation`	`BlockedKFold`	or `WalkForward`
`validation_folds`, `validation_gap`	5, 0	K and g
`one_standard_error_rule`	false	sparser model within 1 SE
`bootstrap_samples`, `bootstrap_block_size`	0, 1	B (0 = off) and ℓ
`confidence_level`, `bootstrap_seed`	0.95, 42	intervals and seed
`fit_intercept`, `standardize`	true, true	see §2.2
`tolerance`, `max_iter`	10^{-7}, 10^5	solver stopping rule and sweep cap
`num_threads`	0	0 = all cores

Invalid values throw `std::invalid_argument` at construction, with a message naming the field. Python raises `ValueError`.

4.3 Usage from C++

```cpp
#include "nabia/regression/RegressionEngine.h"

auto params = std::make_unique<RegressionEngineParams>();
params->min_factors = 1;
params->max_factors = 4;
params->search = SubsetSearch::Exhaustive;
params->train_fraction = 0.7; // last 30% of the rows out of sample
params->alphas = {0.0, 0.5, 1.0};
params->n_lambdas = 30;
params->validation = ValidationScheme::BlockedKFold;
params->bootstrap_samples = 500;
params->bootstrap_block_size = 20; // keep about a month of dependence

RegressionEngine engine(std::move(params));
// Index-free: y is T, X is T x N (rows = time), names optional.
RegressionComputation r = engine.run(y, X, names);
r.factor_names; r.coefficients; r.alpha; r.lambda; // the selected model
r.best.test_r2; r.bootstrap->lower; r.bootstrap->upper;

// With an index: target and factors must share it (align with drop_missing first).

RegressionResult<Date> rr = engine.run(target_series, factor_frame);
rr.predictions; rr.residuals; rr.test_start;

// A single elastic net, or a whole path.
ElasticNet en(std::make_unique<ElasticNetParams>(ElasticNetParams{.lambda = 1e-4, .
    alpha = 0.5}));
en.fit(X, y);
const ElasticNetFit& fit = en.result(); // coefficients in original units
auto grid = nabia::regression::lambda_grid(nabia::regression::lambda_max(X, y,
    1.0), 60, 1e-4);
```

```
auto path = nabia::regression::elastic_net_path(X, y, ElasticNetParams{.alpha =
    1.0}, grid);
```

4.4 Usage from Python

The module `nabia_python_bridge.regression` exposes the same classes, with `lambda` spelled `lambda_`. Arrays are `float64` numpy arrays, and the computation runs with the GIL released. `frames.regress` adds pandas alignment and labels:

```
import numpy as np
from nabia_python_bridge import regression, frames

# numpy: y (T,), X (T, N)
r = regression.RegressionEngine(min_factors=1, max_factors=4, train_fraction=0.7,
                                search=regression.SubsetSearch.Exhaustive,
                                alphas=[0.0, 0.5, 1.0], n_lambdas=30,
                                bootstrap_samples=500, bootstrap_block_size=20).run(y,
                                     X, names)
r.factor_names, r.coefficients, r.alpha, r.lambda_
r.best.test_r2, r.validation_curve.mse, r.bootstrap.lower, r.bootstrap.upper

# In sample, chosen by BIC
regression.RegressionEngine(max_factors=5, train_fraction=1.0,
                            in_sample_criterion=regression.InSampleCriterion.BIC).run
                                (y, X, names)

# pandas: aligned on the common index, rows with a missing value dropped
out = frames.regress(fund_returns, factor_returns, min_factors=1, max_factors=4,
                     validation="WalkForward", bootstrap_samples=500,
                         bootstrap_block_size=20)
out.coefficients # Series indexed by factor name
out.bootstrap # DataFrame: mean, std_error, lower, upper, selection_frequency
out.best_by_size # DataFrame: one row per subset size
out.predictions, out.residuals, out.test_start

# The elastic net itself
lmax = regression.lambda_max(X, y, alpha=1.0)
path = regression.elastic_net_path(X, y, regression.ElasticNetParams(alpha=1.0),
                                   regression.lambda_grid(lmax, 60, 1e-4))
```

4.5 Worked example: recovering a known factor mix

`examples/01_factor_replication.py` builds a "fund" from real daily returns (2012–2026):

$$\text{fund}_t = 0.0001 + 0.50\,\text{NDX}_t + 0.30\,\text{Gold}_t + 0.20\,10\text{y}_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}\big(0, (0.02/\sqrt{252})^2\big).$$

It then lets the engine choose 1 to 4 factors out of 14 candidates. The candidates include close substitutes: the S&P 500 ($\rho = 0.93$ with the Nasdaq), silver, and 5y/30y bonds. With a 70/30 split, 3 alphas, 30 lambdas, 5 blocked folds and 500 bootstrap samples in 20-day blocks:

Factor	True	Estimate	95% bootstrap interval
Nasdaq 100	0.50	0.497	[0.493, 0.500]
Gold	0.30	0.300	[0.295, 0.305]
10y notes	0.20	0.193	[0.175, 0.212]
Copper (not in the fund)	0	0.003	[−0.002, 0.007]

- The test R^2 is 0.981, and the out-of-sample tracking error is 1.98%/yr against the 2% noise.
- Copper wins the fourth slot on the test set by noise (the remark of §3.5), and its interval contains zero.

The script plots the lasso path, the best R^2 by subset size, the validation curve with its standard errors, the bootstrap intervals against the true weights, and the cumulative fund against its replication.

5 Verification

`tests/test_regression.cpp` (Catch2) checks:

- $\lambda = 0$ reproduces OLS;
- $\alpha = 0$ reproduces the ridge closed form $(\tilde{X}^\top \tilde{X} + n\lambda I)^{-1} \tilde{X}^\top \tilde{y}$;
- solutions satisfy the optimality conditions (3);
- $\lambda_{\max}$ zeroes every coefficient exactly, the next point of the grid does not, and the grid runs from $\lambda_{\max}$ to $r\lambda_{\max}$;
- constant factors get a zero coefficient;
- exhaustive and forward search recover the true factors out of sample, and Auto switches to Forward above the limit;
- in-sample BIC rejects noise factors;
- a single-point grid skips validation;
- walk-forward validation works with the 1-SE rule;
- bootstrap intervals cover the true coefficients;
- results, bootstrap draws included, are identical for 1 and 4 threads;
- refitting keeps the out-of-sample predictions;
- the `DataFrame` interface keeps names and the index;
- invalid parameters and data throw.

`python/tests/bridge/test_regression.py` repeats the key checks through Python. The suites also run under AddressSanitizer.

6 Limitations

- The model is linear with constant coefficients. Time-varying exposures need rolling windows, or a state-space model such as the Kalman filter of the forecasting module.
- The bootstrap is conditional on the selected subset and hyperparameters (§3.7).
- Exhaustive search grows as $\binom{N}{k}$. Forward search is greedy.
- The test error of the selected model is optimistic when many subsets are compared.

References

- J. Friedman, T. Hastie, R. Tibshirani (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software* 33(1).
- H. Zou, T. Hastie (2005). Regularization and variable selection via the elastic net. *JRSS B* 67(2).
- H. Zou, T. Hastie, R. Tibshirani (2007). On the "degrees of freedom" of the lasso. *Annals of Statistics* 35(5).
- P. Tseng (2001). Convergence of a block coordinate descent method for nondifferentiable minimization. *JOTA* 109(3).
- B. Efron (2004). The estimation of prediction error: covariance penalties and cross-validation. *JASA* 99(467).
- H. Künsch (1989). The jackknife and the bootstrap for general stationary observations. *Annals of Statistics* 17(3).
- J. Campbell, S. Thompson (2008). Predicting excess stock returns out of sample. *Review of Financial Studies* 21(4).
- T. Hastie, R. Tibshirani, J. Friedman (2009). *The Elements of Statistical Learning*, 2nd ed., Springer.